Perfect. Now we'll move from **"what happens"** to **"why it happens"**.

You've already understood the flow. The next step is to understand **what code is running inside the Linux kernel** when your application calls `send()`.

We'll focus on **Transport Layer (Layer 4)** because it is the heart of Linux socket programming and one of the most common interview topics.

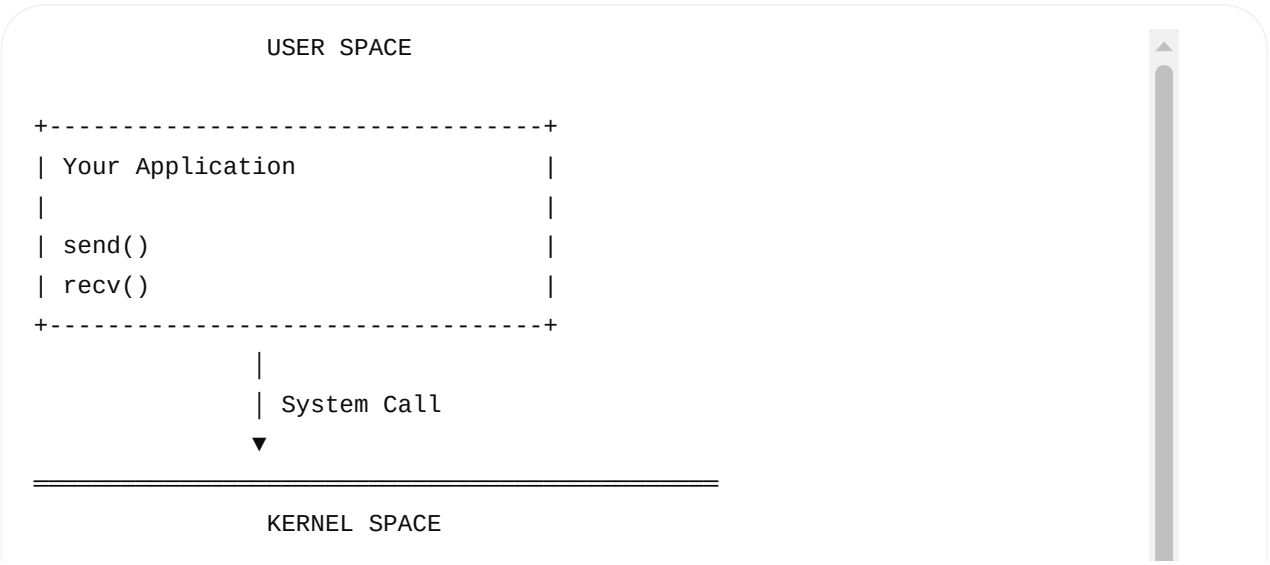
Day 2 – Part 3: Transport Layer (Layer 4)

Learning Objectives

After this lesson, you should understand:

- Why the Transport layer exists
- TCP vs UDP
- Why segmentation is needed
- TCP header fields
- Port numbers
- TCP three-way handshake
- Sequence & ACK numbers
- Flow control
- Retransmission
- How `send()` maps to the kernel's TCP stack
- Where Linux kernel executes all of this

Where is the Transport Layer?



```
═══════════════════════════════════════════

+----------------------------------+
| Transport Layer (TCP / UDP)      |
+----------------------------------+
               |
               ▼
+----------------------------------+
| Network Layer (IP)               |
+----------------------------------+
               |
               ▼
+----------------------------------+
| Data Link (Ethernet)             |
+----------------------------------+
```

Notice

Your program never builds a TCP header.

The Linux kernel does.

Why Do We Need Transport Layer?

Imagine you want to send a book.

The book has

```
500 Pages
```

Can the courier carry the whole book in one envelope?

No.

Instead

```
Book

↓

Split

↓

Envelope 1

Envelope 2

Envelope 3
```

```
Envelope 4
```

Networking is exactly the same.

Without Transport Layer

Suppose

```
Application

↓

10 MB Data

↓

Ethernet
```

Impossible.

Ethernet has frame size limits.

Therefore

Transport Layer divides data.

Segmentation

Application

```
send(sockfd,buffer,5000,0);
```

Application thinks

```
5000 Bytes
```

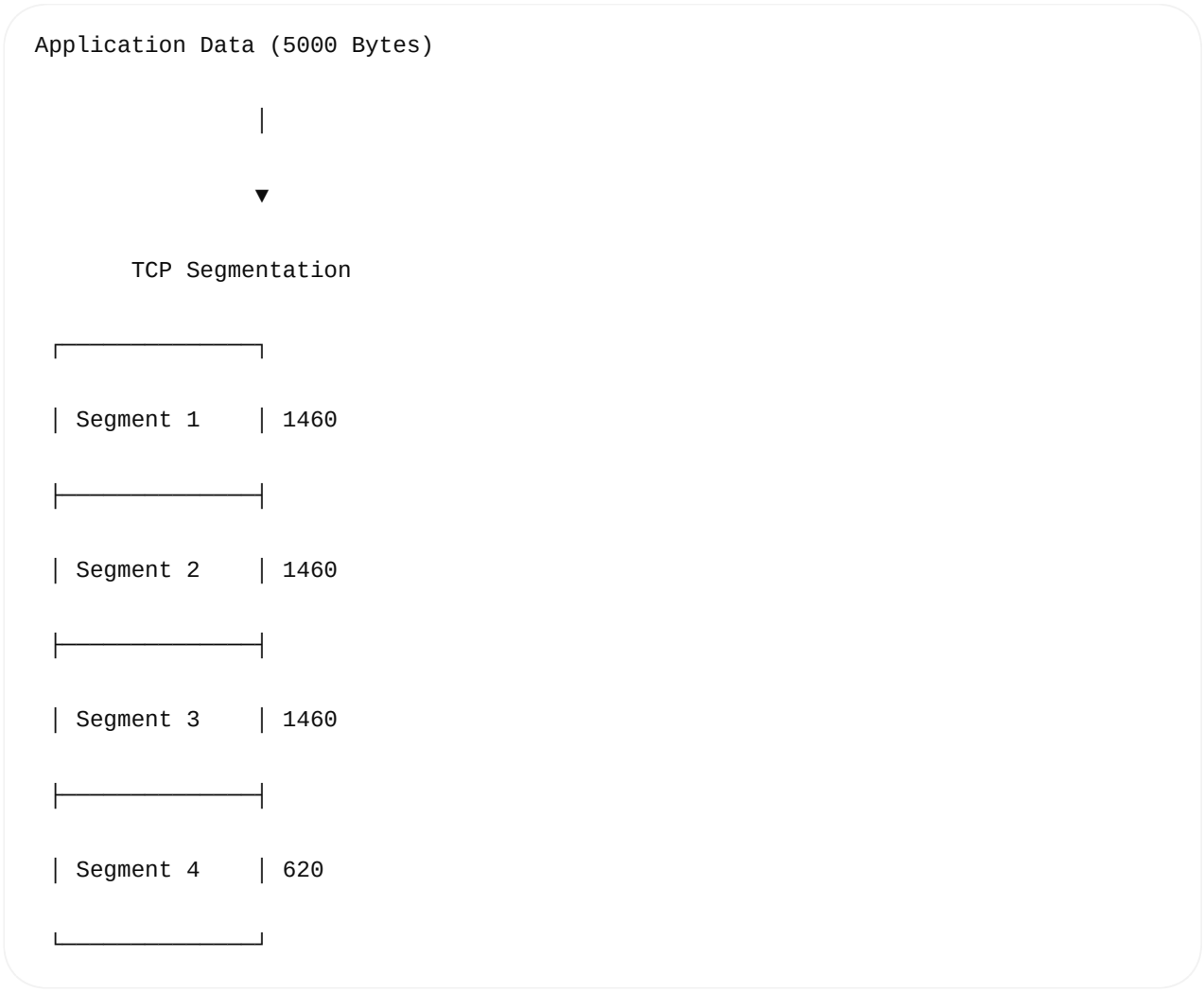
Kernel thinks

```
Maximum Segment Size

1460 Bytes
```

So kernel performs

```
5000 Bytes

↓

1460

1460

1460

620
```

Visualization



Each segment is sent independently.

TCP Header

Every segment receives its own TCP Header.

Visualization

```
+-----------------------------------------------+

TCP Header

-------------------------------------------------

Source Port

Destination Port

Sequence Number

ACK Number

Flags

Window Size

Checksum

-------------------------------------------------

Application Data

+-----------------------------------------------+
```

Port Numbers

Think of IP Address as an apartment building.

Example

```
192.168.1.10
```

But many people live inside.

How do we know which apartment?

Port Number.

Visualization

```
Computer

IP = 192.168.1.10

        |
```

```
        ▼

+--------------------------+

Chrome

Port 52000

+--------------------------+

SSH

Port 22

+--------------------------+

Web Server

Port 80

+--------------------------+

DNS

Port 53

+--------------------------+
```

Port identifies the application.

Source Port

Kernel chooses

```
52000
```

Example

```
Chrome

↓

Source Port

52000
```

Destination Port

Depends on protocol.

```
HTTPS

↓

443
```

Kernel inserts

```
Source Port = 52000

Destination Port = 443
```

Interview Question

Why do we need ports?

Answer

One computer runs many applications.

Ports identify the correct application.

Sequence Number

Suppose

Application sends

```
ABCDEFGHIJ
```

Kernel divides

```
ABC

DEF

GHI

J
```

TCP assigns

```
Seq = 1

Seq = 4

Seq = 7

Seq = 10
```

Visualization

```
Segment 1

ABC

Seq 1

↓

Segment 2

DEF

Seq 4

↓

Segment 3

GHI

Seq 7

↓

Segment 4

J

Seq 10
```

Now receiver knows the order.

ACK Number

Receiver receives

```
ABC
```

Replies

```
ACK = 4
```

Meaning

"I received bytes 1–3, send byte 4 next."

Visualization

```
PC-A

Seq = 1

──────────►

PC-B

ACK = 4

◄──────────
```

Three-Way Handshake

Before sending data

TCP creates a connection.

Visualization

```
Client (PC-A)                    Server (PC-B)

      connect()

          |

          | SYN
──────────►

          |

          |◄──────────
          | SYN + ACK

          |
```

```
 ACK
 ──────────►

Connection Established
```

Meaning

Packet	Meaning
SYN	I want to connect
SYN+ACK	I agree
ACK	Let's communicate

What Executes This?

Your code

```
connect(sockfd,...);
```

Kernel performs

```
SYN

↓

Wait

↓

SYN ACK

↓

ACK
```

Application never creates these packets.

Retransmission

Suppose

```
Segment 2

Lost
```

Visualization

```
Segment 1

──────►

Segment 2

XXXXXXXX Lost

Segment 3

──────►
```

Receiver

never sends ACK.

Kernel waits.

Timeout.

Kernel automatically retransmits.

```
Segment 2

──────►
```

Application never calls

```
send()

again
```

Kernel handles it.

Sliding Window

Instead of sending one packet at a time,

TCP sends multiple packets.

Visualization

```
Window Size = 3

PC-A

Pkt1

Pkt2

Pkt3

──────────►

PC-B

ACK

◄──────────
```

This improves performance.

TCP Flags

Flag	Meaning
SYN	Start connection
ACK	Acknowledge data
FIN	Close connection
RST	Reset connection
PSH	Push data immediately
URG	Urgent data

TCP Header (Simplified)

```
0                   15 16                  31
+---------------------+---------------------+
| Source Port         | Destination Port    |
+---------------------+---------------------+
| Sequence Number                           |
```

```
+---------------------------------------------+
| Acknowledgment Number                       |
+---------------------------------------------+
| HLEN |Flags| Window Size                     |
+---------------------------------------------+
| Checksum | Urgent Pointer                  |
+---------------------------------------------+
| Application Data ...                        |
+---------------------------------------------+
```

How `send()` Travels

```
Application

send(5000)

       |

       ▼

System Call

       |

       ▼

Linux Kernel

       |

       ▼

TCP

↓

Segment 1

↓

Segment 2

↓

Segment 3

↓

Segment 4
```

```
        |
        ▼
IP Layer
        |
        ▼
Ethernet
        |
        ▼
NIC
```

Remember:

Each segment becomes its own IP packet and Ethernet frame.

Linux APIs Related to Transport Layer

API	Purpose
`socket()`	Create TCP/UDP endpoint
`bind()`	Bind socket to a local IP and/or port
`listen()`	Mark a TCP socket as passive (server)
`accept()`	Accept a new client connection
`connect()`	Initiate a TCP connection (starts the three-way handshake)
`send()` / `write()`	Send data
`recv()` / `read()`	Receive data
`close()`	Close the socket (TCP FIN exchange for a normal close)

Linux Commands

Observe connections:

```
ss -t
```

Show listening sockets:

```
ss -ltn
```

Capture TCP traffic:

```
sudo tcpdump -i eth0 tcp
```

Capture a handshake:

```
sudo tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn|tcp-ack) != 0'
```

Interview Questions

Q1. Why is segmentation required?

Because lower-layer technologies such as Ethernet have frame size limits. TCP divides large application data into manageable segments.

Q2. Who performs TCP segmentation?

The Linux kernel's TCP stack.

Q3. Does the application create TCP headers?

No. The application provides payload data through socket APIs. The kernel constructs the TCP header.

Q4. Who performs retransmissions?

The Linux kernel's TCP implementation.

Q5. Who chooses the source port?

Usually the kernel automatically assigns an ephemeral source port unless the application explicitly binds a local port using `bind()`.

Next Lesson (Part 4)

We'll move to **Layer 3 – Network Layer**, where you'll learn:

- IPv4 header field by field
- Routing table lookup
- Longest Prefix Match (how Linux selects a route)
- Default gateway
- TTL and why it decreases at every router
- Packet fragmentation and MTU
- What happens when the destination is **on the same LAN** versus **on a different network**
- Why ARP is used for a gateway instead of the final destination in remote communication
- Linux commands: `ip route`, `ip addr`, `ping`, `traceroute`, and packet captures showing IP behavior

This is the point where the packet leaves the TCP layer and begins its journey across the network.

Yes. This is a **very important concept**, and the easiest way to understand it is to separate **IP's job (Layer 3)** from **MAC's job (Layer 2)**.

I'll explain **Same LAN vs Different Network** step-by-step.

---

1. First: IP vs MAC — What Is the Difference?

Think of sending a parcel.

```
IP address = Final destination address
MAC address = Next local delivery address
```

More precisely:

- **IP address** is used by Layer 3 to determine the destination network and route.
- **MAC address** is used by Layer 2 to deliver the Ethernet frame to the **next hop on the local LAN**.

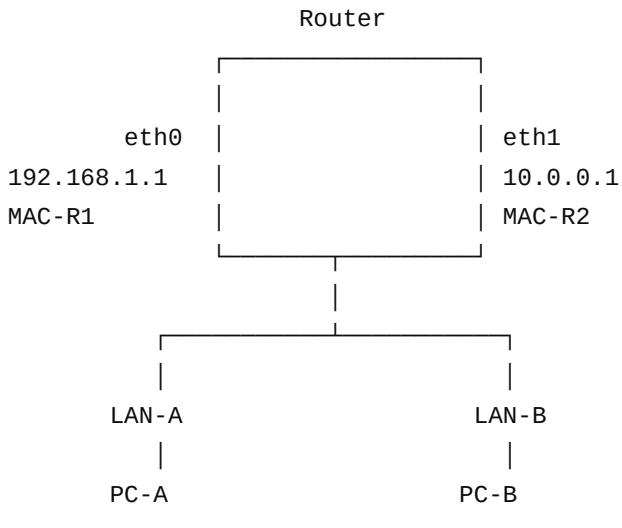
The crucial point is:

> **The destination IP usually remains the final destination IP end-to-end, while the destination MAC changes at every Layer-3 hop.**

---

2. Example Network

Let's create a simple network.



```
          |                    |
IP:192.168.1.10            IP:10.0.0.20
MAC:A-A-A-A                MAC:B-B-B-B
```

There are **two different networks**:

```
LAN-A = 192.168.1.0/24

LAN-B = 10.0.0.0/24
```

---

3. Case 1 — Same LAN

Let's first ignore the router.

Suppose:

```
PC-A
IP  = 192.168.1.10
MAC = AA-AA-AA

        |
        | Switch
        |
        ▼

PC-B
IP  = 192.168.1.20
MAC = BB-BB-BB
```

Both are:

```
192.168.1.0/24
```

So they are on the **same IP network**.

---

4. PC-A Wants to Send Data to PC-B

Application:

```c
send(sockfd, data, len, 0);
```

First TCP does its work.

```
Application
     |
     ▼
TCP
     |
     ▼
TCP Segment
```

Then Layer 3 gets the segment.

---

5. Layer 3 Checks Destination IP

PC-A sees:

```
Source IP      = 192.168.1.10
Destination IP = 192.168.1.20
```

The kernel checks its routing table.

Conceptually:

```
Destination = 192.168.1.20

Local route:
192.168.1.0/24 → eth0
```

It asks:

> "Is the destination on my local network?"

Answer:

```
YES
```

Therefore:

> **Send directly to PC-B.**

There is no router involved for this local delivery.

---

6. Layer 2 Now Needs MAC

Here's the important transition.

Layer 3 knows:

```
Destination IP
       ↓
192.168.1.20
```

But Ethernet needs:

```
Destination MAC
       ↓
????????????
```

So Linux checks its ARP/neighbor cache:

```
IP                   MAC
-------------------------------
192.168.1.20         BB-BB-BB
```

Now Linux knows:

```
192.168.1.20
      ↓
BB-BB-BB
```

---

7. Ethernet Frame

Linux creates:

```
┌──────────────────────────────────────┐
| Ethernet Header                      |
|                                      |
| Destination MAC = BB-BB-BB           |
| Source MAC      = AA-AA-AA           |
|                                      |
├──────────────────────────────────────┤
| IP Packet                            |
|                                      |
| Destination IP = 192.168.1.20        |
| Source IP      = 192.168.1.10        |
|                                      |
└──────────────────────────────────────┘
```

Notice something extremely important:

```
Layer 2:

Destination MAC = PC-B
```

```
Source MAC      = PC-A


Layer 3:

Destination IP = PC-B
Source IP      = PC-A
```

Everything points directly to PC-B because PC-B is on the same LAN.

---

8. Switch Receives the Frame

```
PC-A
 |
 |
 | Destination MAC = BB-BB-BB
 ▼
┌──────────────┐
|   SWITCH     |
└──────┬───────┘
       |
       | Port 2
       ▼
     PC-B
```

Switch checks:

```
Destination MAC = BB-BB-BB
```

MAC table:

```
BB-BB-BB → Port 2
```

So:

```
Port 1
  |
  ▼
Switch
  |
  ▼
Port 2
  |
  ▼
PC-B
```

Done.

9. Same LAN — Complete Flow

Key point:

Same LAN → ARP for the destination host's MAC.

---

10. Now Different Network

Now change PC-B:

```
PC-A                          PC-B

IP = 192.168.1.10             IP = 10.0.0.20
MAC = AA-AA-AA                MAC = BB-BB-BB

192.168.1.0/24                10.0.0.0/24
       |                             |
       |                             |
       └──────── Router ─────────────┘
```

Now PC-A wants:

```
10.0.0.20
```

PC-A checks:

```
192.168.1.10

Destination:

10.0.0.20
```

Are they in the same network?

```
192.168.1.0/24
        vs
10.0.0.0/24
```

No.

---

11. Layer 3 Says "Use Router"

PC-A's routing table might contain:

```
Destination        Gateway          Interface

192.168.1.0/24     directly connected eth0

default            192.168.1.1     eth0
```

Destination:

```
10.0.0.20
```

doesn't match the local network route.

So Linux uses:

```
Default Gateway

192.168.1.1
```

---

12. Very Important Question

Does PC-A ARP for:

```
10.0.0.20 ?
```

× No.

PC-A cannot directly reach that IP at Layer 2.

Instead:

```
PC-A ARPs for:

192.168.1.1
```

which is the router's local interface.

---

13. ARP

PC-A asks:

```
Who has

192.168.1.1?
```

Router replies:

```
192.168.1.1

MAC = RR-RR-RR
```

PC-A stores:

```
192.168.1.1
       ↓
RR-RR-RR
```

---

14. Now Look Carefully at the Packet

PC-A creates an IP packet:

```
┌──────────────────────────────────┐
| IP Header                        |
|                                  |
| Source IP      = 192.168.1.10    |
| Destination IP = 10.0.0.20       |
|                                  |
└──────────────────────────────────┘
```

Destination IP is still PC-B.

But Ethernet frame is:

```
┌──────────────────────────────────────┐
| Ethernet Header                      |
|                                      |
| Destination MAC = RR-RR-RR           |
| Source MAC      = AA-AA-AA           |
|                                      |
├──────────────────────────────────────┤
| IP Packet                            |
|                                      |
| Destination IP = 10.0.0.20           |
| Source IP      = 192.168.1.10        |
└──────────────────────────────────────┘
```

This is the most important visualization.

---

15. IP Says "PC-B"

```
IP:

192.168.1.10
      |
      |
      ▼
10.0.0.20
```

But Ethernet says:

```
MAC:

AA-AA-AA
      |
      |
      ▼
RR-RR-RR
```

Why?

Because:

> **The router is the next Layer-2 hop.**

---

16. PC-A → Switch → Router

```
PC-A
 |
 | Ethernet
 |
 | Destination MAC = Router
 | Destination IP  = PC-B
 ▼
SWITCH
 |
 ▼
ROUTER
```

The switch only cares about:

```
Destination MAC = RR-RR-RR
```

It doesn't care that:

```
Destination IP = 10.0.0.20
```

for ordinary Layer-2 forwarding.

---

17. Router Receives the Frame

Router receives:

```
Ethernet Frame

Dst MAC = RR-RR-RR
Src MAC = AA-AA-AA

        ↓

IP Packet

Dst IP = 10.0.0.20
Src IP = 192.168.1.10
```

Router checks the Ethernet destination MAC.

It's for the router.

So router removes the Ethernet header and processes the IP packet.

---

18. Router Performs Routing

Router checks its routing table.

```
Destination:

10.0.0.20

        ↓

10.0.0.0/24

        ↓

eth1
```

Router says:

> "PC-B is reachable through my eth1 interface."

---

19. Router Needs PC-B's MAC

Now we have entered another Layer-2 network.

```
Router eth1
10.0.0.1
MAC = R2-R2-R2
```

```
        |
        | Switch
        |
        ▼

PC-B
10.0.0.20
MAC = BB-BB-BB
```

Router needs:

```
10.0.0.20 → BB-BB-BB
```

So router uses ARP on **that network**.

---

20. Router Creates a NEW Ethernet Frame

This is very important.

The IP packet is forwarded, but the Layer-2 frame is rebuilt for the next link.

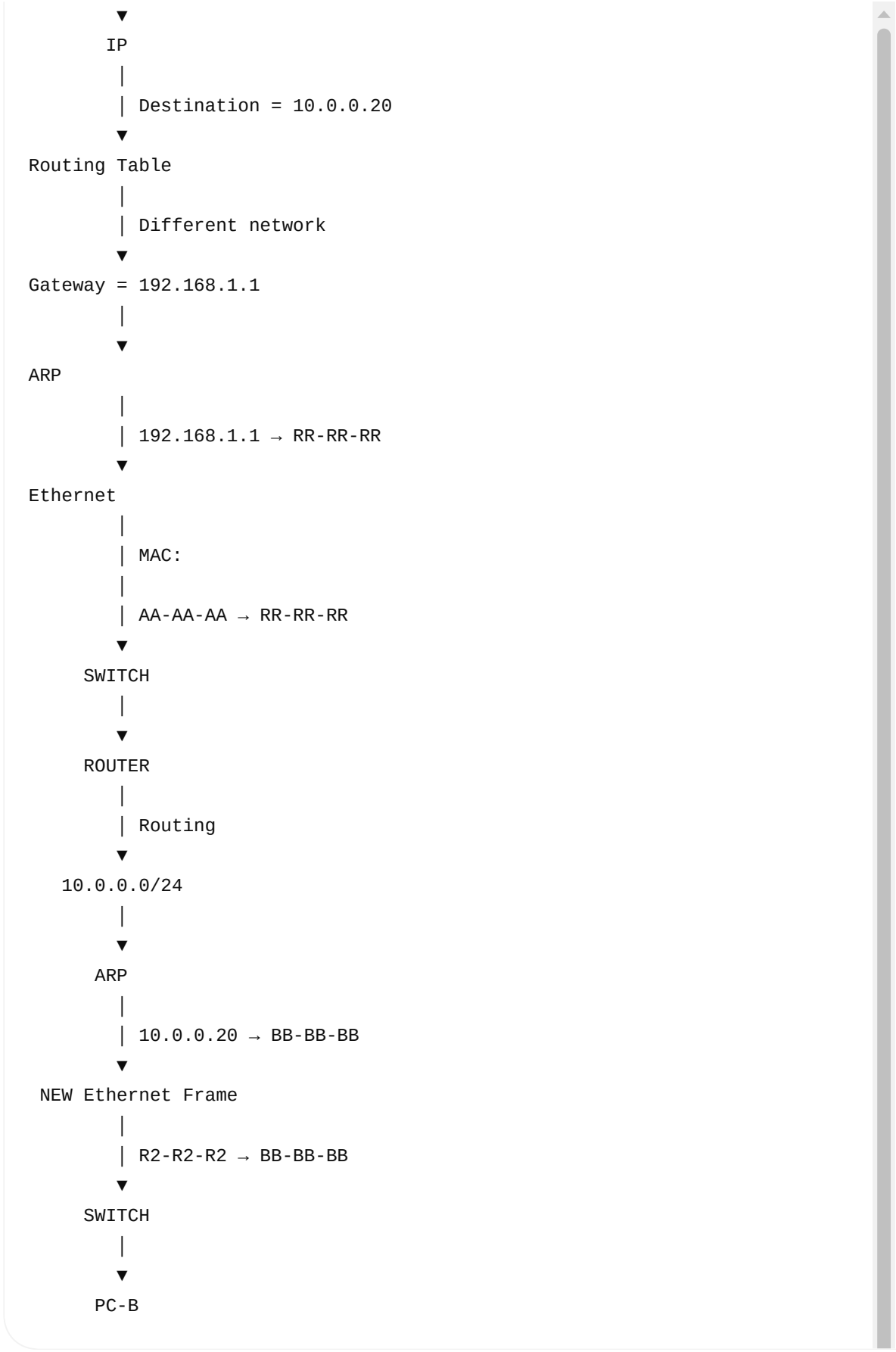
```
NEW Ethernet Frame

Destination MAC = BB-BB-BB
Source MAC      = R2-R2-R2

        |
        ▼

IP Packet

Destination IP = 10.0.0.20
Source IP      = 192.168.1.10
```

So the MAC addresses changed.

---

21. Complete Different-Network Flow

```
PC-A
192.168.1.10
MAC = AA-AA-AA
        |
        |
        ▼
      TCP
        |
```



---

22. The Most Important Concept

Look at this table:

Hop	Source MAC	Destination MAC	Source IP	Destination IP
PC-A → Router	PC-A	Router	PC-A	PC-B
Router → PC-B	Router	PC-B	PC-A*	PC-B

*Ignoring NAT for this explanation.

Notice:

MAC addresses

Change at every Layer-3 hop.

```
PC-A MAC
   ↓
Router MAC

then

Router MAC
   ↓
PC-B MAC
```

IP addresses

Normally remain:

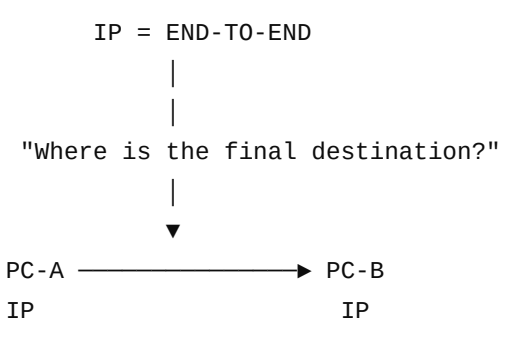
```
PC-A IP
   ↓
PC-B IP
```

from source to destination, unless something like NAT changes them.

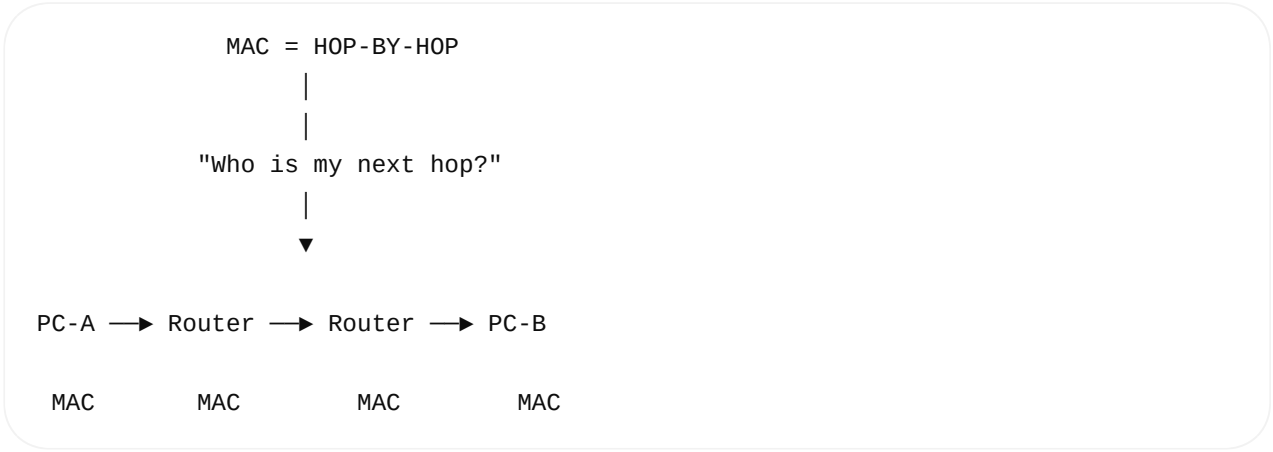
---

23. This Is the Best Mental Model

Remember:



But:



For a same-LAN destination, the next hop **is the destination host itself**.

---

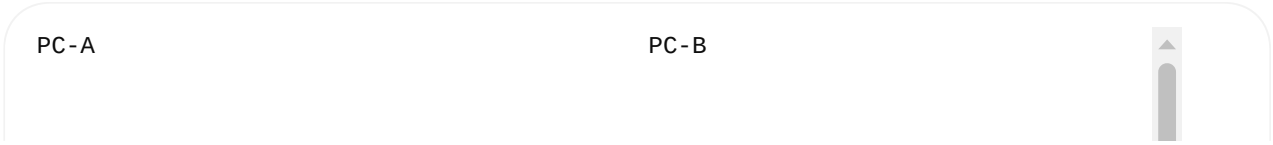
24. Same LAN vs Different Network

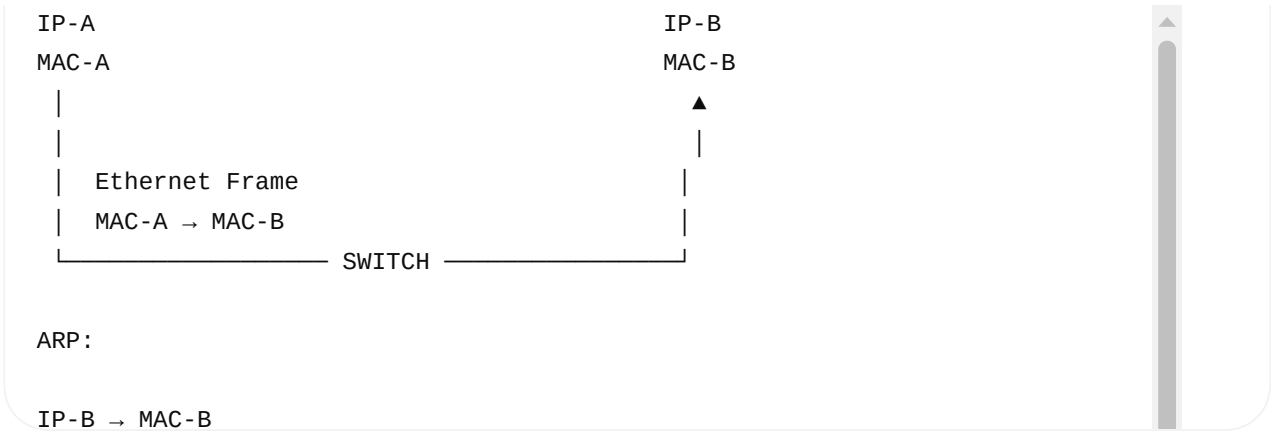
	Same LAN	**Different Network**
Destination IP	PC-B	PC-B
Routing decision	Local/direct route	Router/default gateway
ARP target	**PC-B**	**Next-hop router**
First Ethernet destination MAC	PC-B	Router
Switch used?	Usually yes	Usually yes
Router involved?	× No	✅ Yes
MAC changes?	No router hop, so same local src/dst pair	Changes at every router hop
IP destination	PC-B	PC-B
Layer 2 destination	PC-B	Next hop

---

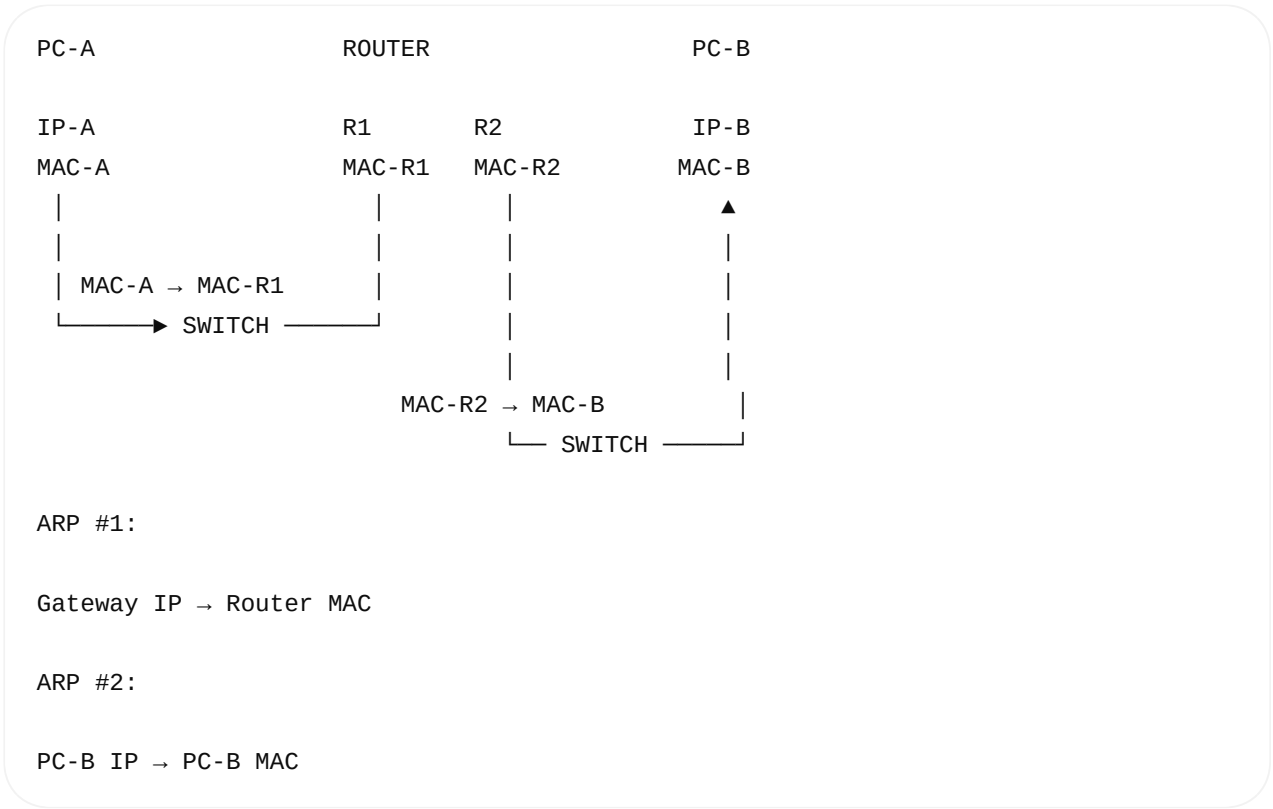
25. One Diagram to Memorize for Interviews

Same LAN





Different Network



---

26. Interview Question You Should Be Ready For

Q: PC-A wants to send data to PC-B. How does it know whether to send directly to PC-B or through a router?

Answer:

> The Layer-3 IP stack compares the destination IP address with the routes in the routing table. If the destination matches a directly connected/local network, the host sends the packet directly to the destination host and uses ARP to resolve that host's MAC address. If the destination is on another network, the host selects a next-hop router from the routing table and uses ARP to resolve the router's MAC address. The

> IP destination remains the remote host, while the Ethernet destination MAC is the next hop.

That is a **strong interview answer**.

The 3 Questions to Ask Yourself

Whenever you see a networking problem, go through these three questions:

```
1️⃣ LAYER 3 — IP

"Is the destination on my network?"

        │
        ├── YES ──▶ Direct
        │
        └── NO ──▶ Router


2️⃣ ARP

"Who is my next hop?"

        │
        ├── Same LAN ──▶ Destination MAC
        │
        └── Remote ──▶ Gateway MAC


3️⃣ LAYER 2 — MAC

"Which local device should receive
this Ethernet frame?"

        │
        ▼
Destination MAC
        │
        ▼
Switch MAC table
        │
        ▼
Output port
```

If you understand these **three questions**, the relationship between **IP, MAC, ARP, switches, and routers** becomes much easier.

So the routher only do the rework in Layer 2 only not above layers. Its right

Almost right — but there is an important correction.

The router does not only rework Layer 2

A router primarily makes a **Layer 3 (IP) forwarding decision**, and it **rebuilds the Layer 2 frame** for the next network.

Think of it like this:

```
        ROUTER
          |
          |
   ┌──────▼──────┐
   | Layer 3     |  ← Router makes forwarding decision
   | IP Routing  |
   └─────────────┘
          |
          |
   ┌──────▼──────┐
   | Layer 2     |  ← Old Ethernet frame removed
   | Ethernet    |     New Ethernet frame created
   └─────────────┘
```

What happens at the router?

Suppose:

```
PC-A                          PC-B

192.168.1.10                  10.0.0.20
     |                            ▲
     |                            |
     └──────── Router ────────────┘
```

Frame arriving at router

```
Ethernet Frame

Src MAC = PC-A
Dst MAC = Router
       |
       ▼
IP Packet
```

```
Src IP = 192.168.1.10
Dst IP = 10.0.0.20
```

The router:

1. Removes Layer-2 information

```
PC-A MAC → Router MAC

        ↓

Ethernet header removed
```

2. Examines Layer 3

Router looks at:

```
Destination IP
        ↓
10.0.0.20
        ↓
Routing table
        ↓
10.0.0.0/24 → eth1
```

So the router decides:

> "Send this packet through eth1."

3. Creates a NEW Layer-2 frame

```
NEW Ethernet Frame

Src MAC = Router's eth1 MAC
Dst MAC = PC-B MAC

        +

Same IP packet

Src IP = 192.168.1.10
Dst IP = 10.0.0.20
```

So:

```
BEFORE ROUTER

MAC:
PC-A ──────────▶ Router
```

```
IP:
PC-A ──────────────────────────► PC-B


AFTER ROUTER

MAC:
Router ──────────► PC-B

IP:
PC-A ──────────────────────────► PC-B
```

That's the key concept.

What about Layers 4–7?

For **normal IP forwarding**, the router generally does **not** need to process TCP/UDP/application payloads.

```
Layer 7  Application ──┐
Layer 6  Presentation ─┤
Layer 5  Session ──────┤
Layer 4  TCP/UDP ──────┤  ← normally forwarded unchanged
Layer 3  IP ───────────┤  ← ROUTER PROCESSES THIS
Layer 2  Ethernet ─────┤  ← OLD FRAME → NEW FRAME
Layer 1  Physical ─────┘
```

So for a normal router:

> **Layer 3 is the main decision-making layer. Layer 2 is rebuilt for every outgoing link.**

But there are exceptions

Modern routers can perform functions that inspect higher layers, such as:

- NAT
- Firewall / ACL inspection
- QoS
- TCP/UDP-aware filtering
- Load balancing
- Application-aware security

For example, with NAT:

```
Before router:

Src IP = 192.168.1.10
Dst IP = 8.8.8.8

        ↓ ROUTER / NAT

After router:

Src IP = Public-IP
Dst IP = 8.8.8.8
```

So don't say:

> × "A router never touches Layer 4 and above."

Better interview statement:

> ✅ **"A traditional router primarily operates at Layer 3. It examines the IP header to make a forwarding decision and rebuilds the Layer-2 frame for the next hop. Normally it forwards the Layer-4 payload without terminating the TCP/UDP connection, although modern routers can perform higher-layer functions such as NAT, firewall inspection, and QoS."**

One sentence to memorize

Switch → mainly Layer 2 → MAC → Port

Router → mainly Layer 3 → IP → Next Hop

TCP → Layer 4 → Port, reliability, sequence, ACK, retransmission

combain the all parts into th esinble one based on the above learning

Yes. Let's combine **everything you've learned so far** into **one complete mental model**.

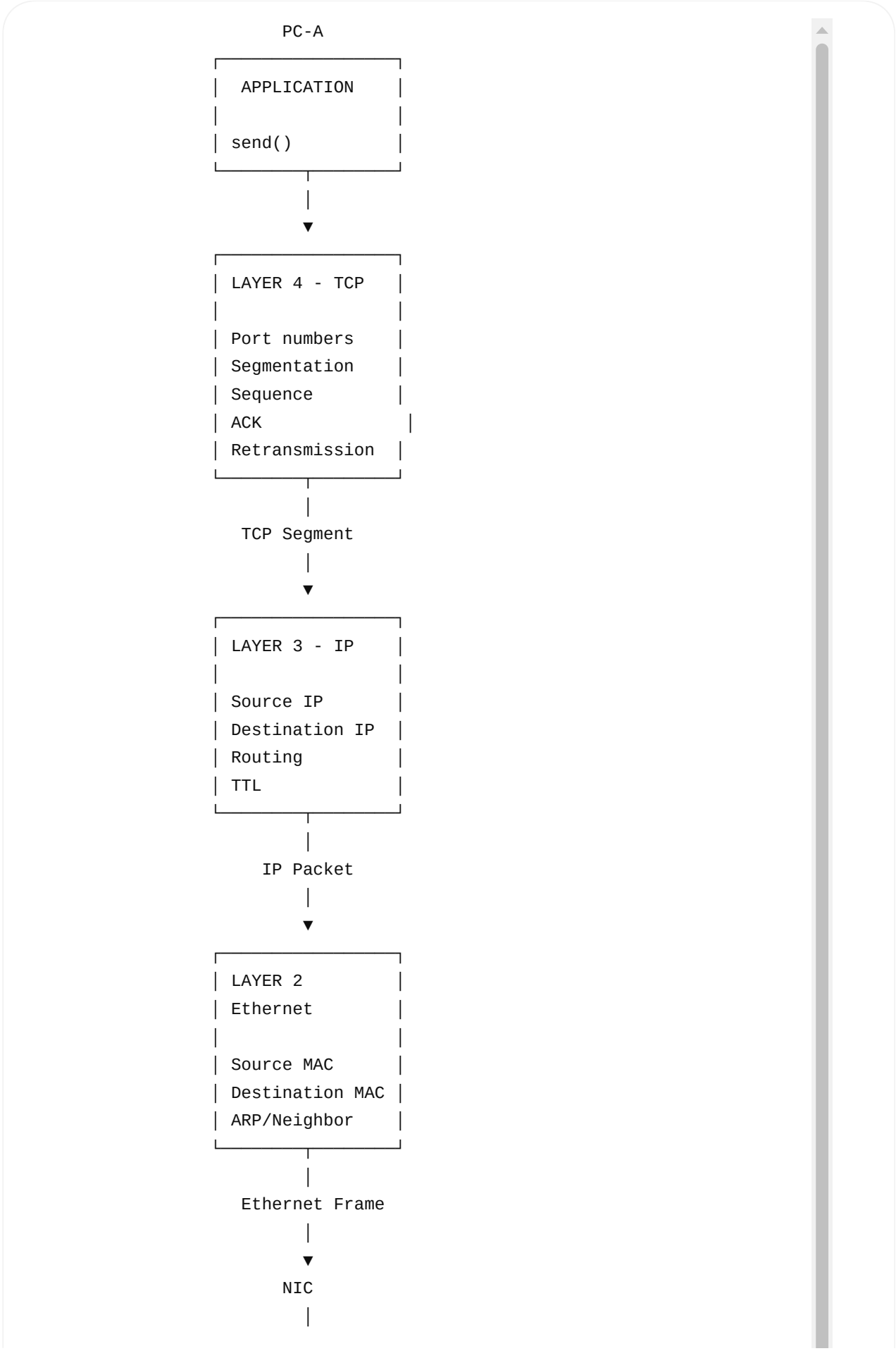
The goal is that when an interviewer asks:

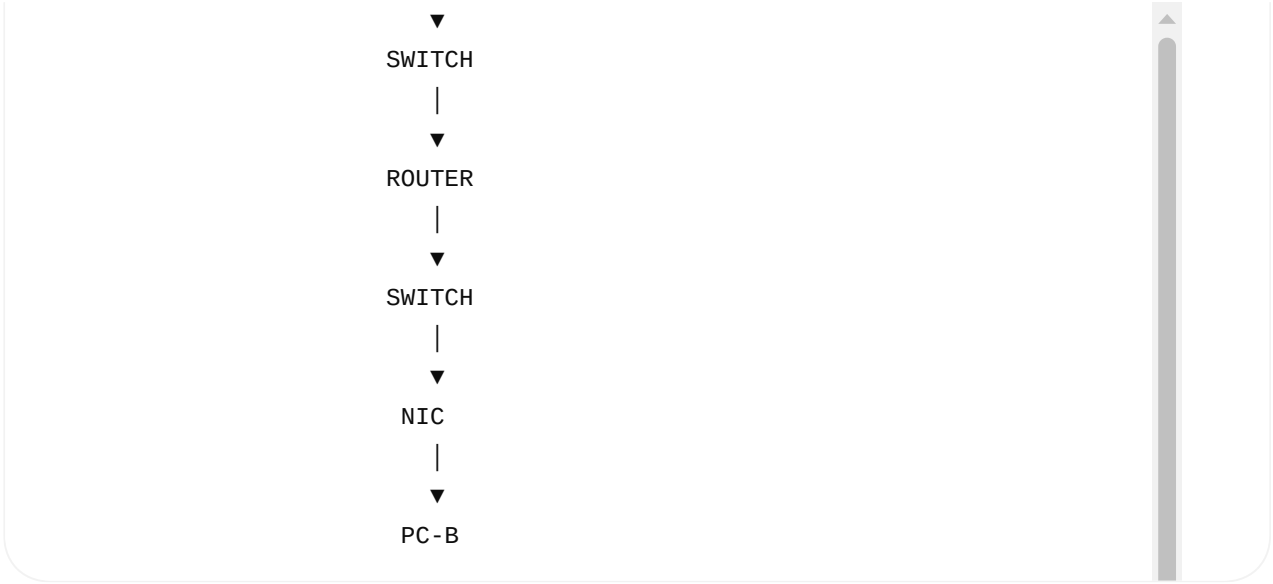
> **"What happens when an application on PC-A sends data to PC-B?"**

you can explain the entire journey from `send()` → TCP → IP → ARP → Ethernet → Switch → Router → PC-B.

Complete Networking Flow — Application to PC-B

1. The Big Picture





But there is an important distinction:

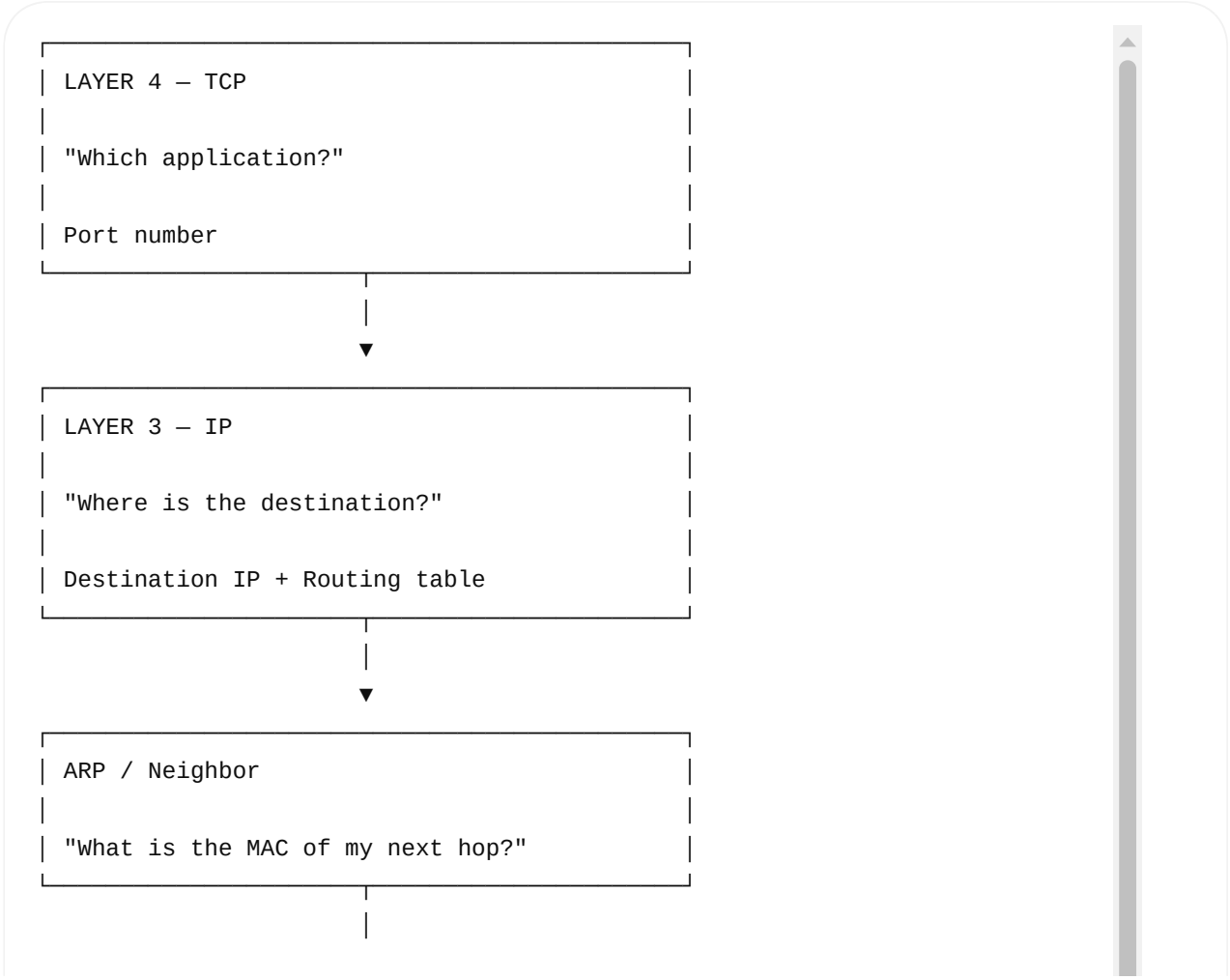
> **If PC-B is on the same LAN, the router is not needed.**

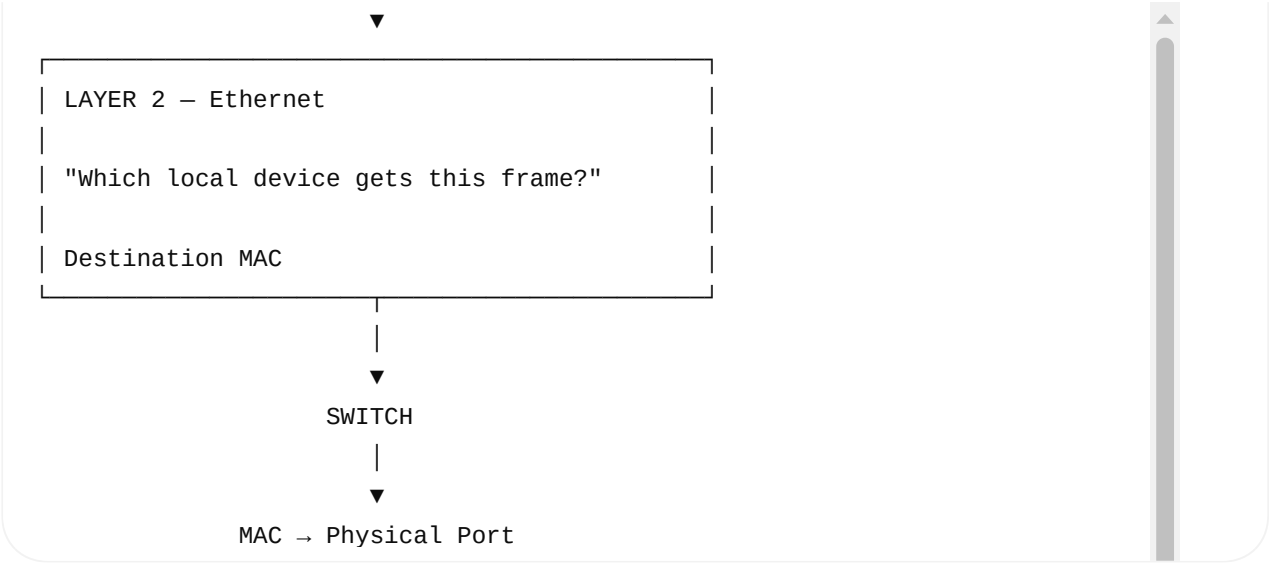
So let's learn both cases.

---

2. First Understand the Four Important Questions

When data moves through a network, think:





This is the foundation.

---

3. Start With an Application

Suppose PC-A runs a client application.

```
send(sockfd, buffer, 5000, 0);
```

The application knows:

```
"I want to send 5000 bytes."
```

It does **not** normally construct:

```
TCP header
IP header
Ethernet header
MAC address
ARP packet
```

The operating system's networking stack handles those tasks.

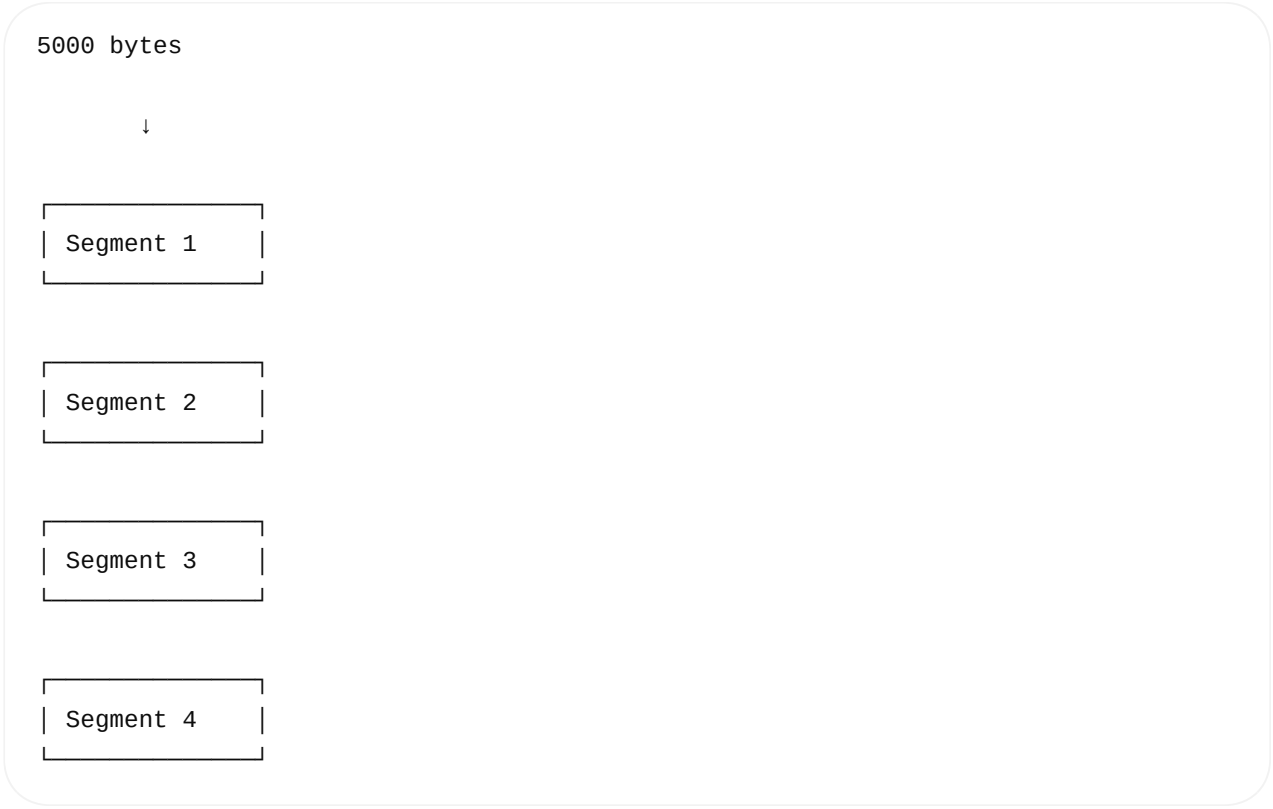
---

4. Application → TCP

The data enters the Transport Layer.

TCP may divide the data into multiple segments.

For example:



Each TCP segment has TCP information such as:

```
Source Port
Destination Port
Sequence Number
ACK Number
Flags
Window
Checksum
```

---

5. TCP's Job

TCP is responsible for things such as:

```
          Retransmission
                |
                ▼
           Flow Control
```

For example:

```
Source Port      = 50000
Destination Port = 443
```

So TCP answers:

> **"Which process/application should receive this data?"**

---

6. TCP Three-Way Handshake

Before a TCP connection is established:

```
PC-A                          PC-B

        SYN
──────────────────►

        SYN + ACK
◄──────────────────

        ACK
──────────────────►

     Connected
```

This establishes the TCP connection.

After that, application data can flow.

---

7. TCP Gives Segment to IP

Now Layer 3 receives the TCP segment.

```
TCP Segment
     |
     ▼
+-----------------------+
| TCP Header            |
+-----------------------+
```

```
| Application Data      |
```

IP adds its own header.

```
+----------------------+
| IP Header            |
+----------------------+
| TCP Header           |
+----------------------+
| Application Data     |
+----------------------+
```

Now we have:

> **IP Packet**

---

8. What Does IP Put in the Header?

Important fields:

```
Source IP
Destination IP
TTL
Protocol
Total Length
```

Example:

```
Source IP      = 192.168.1.10
Destination IP = 192.168.1.20
Protocol       = TCP
TTL            = 64
```

---

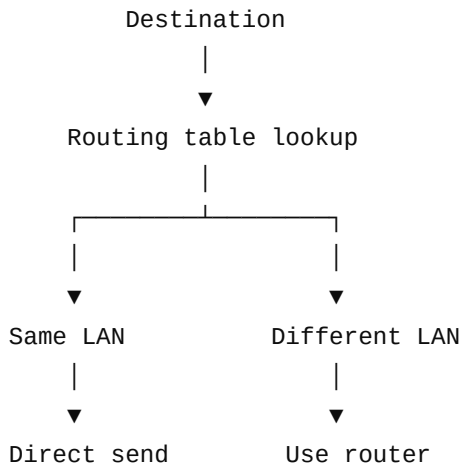
9. Now the Most Important Question

The Linux kernel asks:

> **"Is the destination on my local network?"**

This is determined using the IP address, subnet/prefix, and routing table.

There are two possibilities.



PART A — SAME LAN

10. Example

```
PC-A                                PC-B

IP  = 192.168.1.10                  IP  = 192.168.1.20
MAC = AA-AA-AA                      MAC = BB-BB-BB

        └──────── SWITCH ──────────┘
```

Both belong to:

```
192.168.1.0/24
```

Therefore:

```
Destination is local
```

---

11. ARP on Same LAN

Layer 3 knows:

```
Destination IP

192.168.1.20
```

But Layer 2 needs:

```
Destination MAC

??????????????
```

Linux checks its ARP/neighbor cache.

If it doesn't have the mapping, PC-A sends:

```
Who has 192.168.1.20?
```

using an Ethernet broadcast.

PC-B replies:

```
192.168.1.20
        ↓
BB-BB-BB
```

Linux can then use that MAC for the frame.

---

12. Ethernet Frame — Same LAN

Now Layer 2 creates:

```
┌──────────────────────────────────┐
| Ethernet Header                  |
|                                  |
| Destination MAC = BB-BB-BB       |
| Source MAC      = AA-AA-AA       |
|                                  |
├──────────────────────────────────┤
| IP Packet                        |
|                                  |
| Source IP      = 192.168.1.10    |
| Destination IP = 192.168.1.20    |
|                                  |
├──────────────────────────────────┤
| TCP Segment                      |
├──────────────────────────────────┤
| Application Data                 |
├──────────────────────────────────┤
| FCS                              |
└──────────────────────────────────┘
```

Notice:

```
MAC destination = PC-B
IP destination  = PC-B
```

---

13. Switch

The frame reaches the switch.

Switch looks primarily at:

```
Destination MAC
```

Suppose its table contains:

```
MAC             Port
-----------------------
AA-AA-AA        Port 1
BB-BB-BB        Port 2
```

So:

```
PC-A
  |
  | Port 1
  ▼
SWITCH
  |
  | Port 2
  ▼
PC-B
```

The switch does not need to inspect TCP ports or application data for ordinary Layer-2 forwarding.

---

14. SAME LAN — Final Picture

```
PC-A
|
| Application
|
▼
send()
|
▼
```

```
TCP
|
| Segment
▼
IP
|
| Destination = PC-B
▼
Routing Table
|
| Same network
▼
ARP
|
| PC-B IP → PC-B MAC
▼
Ethernet
|
| PC-A MAC → PC-B MAC
▼
NIC
|
▼
SWITCH
|
| MAC lookup
▼
PC-B
```

Important:

```
Same LAN

ARP target = PC-B
Ethernet destination MAC = PC-B
Router = NOT required
```

PART B — DIFFERENT NETWORK

Now change PC-B.

```
PC-A                            PC-B

192.168.1.10                    10.0.0.20
AA-AA-AA                        BB-BB-BB

       |                           |
```

Networks:

```
PC-A network:

192.168.1.0/24


PC-B network:

10.0.0.0/24
```

They are different.

---

15. IP Routing Decision

PC-A wants:

```
Destination IP = 10.0.0.20
```

Linux checks:

```
ip route
```

Conceptually:

```
192.168.1.0/24 → eth0

default via 192.168.1.1
```

The destination isn't local.

Therefore:

```
Use gateway

192.168.1.1
```

---

16. Very Important: ARP

Does PC-A ask:

```
Who has 10.0.0.20?
```

× No.

It asks:

```
Who has 192.168.1.1?
```

because:

```
192.168.1.1
      ↓
Next-hop router
```

ARP resolves:

```
Gateway IP
     ↓
Gateway MAC
```

Suppose:

```
192.168.1.1
     ↓
RR-RR-RR
```

---

17. First Ethernet Frame

PC-A creates:

```
Ethernet:

Source MAC      = AA-AA-AA
Destination MAC = RR-RR-RR
```

But the IP packet still says:

```
Source IP      = 192.168.1.10
Destination IP = 10.0.0.20
```

So:

```
┌──────────────────────────────────────┐
| Ethernet                             |
|                                      |
| AA-AA-AA → RR-RR-RR                  |
|                                      |
├──────────────────────────────────────┤
```

```
| IP                                    |
|                                       |
| 192.168.1.10 → 10.0.0.20              |
|                                       |
├───────────────────────────────────────┤
| TCP                                   |
├───────────────────────────────────────┤
| Data                                  |
└───────────────────────────────────────┘
```

This is the **critical concept**.

```
MAC says:

"Send to my router."


IP says:

"Ultimately deliver to PC-B."
```

---

18. Switch Sends Frame to Router

```
PC-A
 |
 | Dst MAC = Router
 ▼
SWITCH
 |
 | MAC lookup
 ▼
ROUTER
```

The switch only needs the destination MAC to make its forwarding decision.

---

19. Router Receives the Frame

Router sees:

```
Ethernet:

Dst MAC = Router
Src MAC = PC-A
```

The router accepts the frame.

Then:

```
Ethernet Header
      |
      ▼
Removed
      |
      ▼
IP Packet
```

Now router examines Layer 3.

```
Destination IP
      ↓
10.0.0.20
```

---

20. Router Routing Decision

Router checks its routing table.

```
10.0.0.0/24 → eth1
```

So:

```
Destination = 10.0.0.20

       ↓

Network = 10.0.0.0/24

       ↓

Outgoing Interface = eth1
```

This is why we say:

> **A router primarily operates at Layer 3.**

---

21. Router Needs New MAC Information

Router is now on the second LAN:

```
Router eth1
IP  = 10.0.0.1
```

```
MAC = R2-R2-R2

        |
        |
        ▼

PC-B
IP  = 10.0.0.20
MAC = BB-BB-BB
```

Router needs:

```
10.0.0.20
       ↓
BB-BB-BB
```

It can use its neighbor/ARP cache, or perform ARP if needed.

---

22. Router Creates NEW Ethernet Frame

The router creates a new Layer-2 frame:

```
┌──────────────────────────────────────┐
│ Ethernet                             │
│                                      │
│ Source MAC      = R2-R2-R2           │
│ Destination MAC = BB-BB-BB           │
│                                      │
├──────────────────────────────────────┤
│ IP                                   │
│                                      │
│ Source IP      = 192.168.1.10        │
│ Destination IP = 10.0.0.20           │
│                                      │
├──────────────────────────────────────┤
│ TCP                                  │
├──────────────────────────────────────┤
│ Data                                 │
└──────────────────────────────────────┘
```

Notice:

MAC changed

```
Before router:

AA-AA-AA → RR-RR-RR
```

```
After router:

R2-R2-R2 → BB-BB-BB
```

IP normally remains the same

```
192.168.1.10 → 10.0.0.20
```

for this basic example.

---

23. Why Does MAC Change?

Because MAC is **hop-by-hop**.

```
PC-A
  |
  | MAC
  ▼
Router
  |
  | NEW MAC
  ▼
PC-B
```

Each Ethernet network has its own Layer-2 delivery.

IP provides the Layer-3 path across multiple networks.

---

24. The Most Important Diagram

Memorize this:

```
                LAYER 3
          IP = END-TO-END

 PC-A IP ──────────────────────────────► PC-B IP
192.168.1.10                          10.0.0.20


                LAYER 2
          MAC = HOP-BY-HOP

 PC-A MAC ─────► Router MAC
                    |
                    |
```

```
                    ▼
Router MAC ──────► PC-B MAC
```

Or even simpler:

```
IP:

SOURCE ──────────────────────────────► DESTINATION
       stays logically end-to-end


MAC:

SOURCE ──► NEXT HOP ──► NEXT HOP ──► DESTINATION
       changes at every Layer-3 hop
```

---

25. What Happens to TCP?

Now connect Layer 4.

The router normally does **not** terminate the TCP connection.

```
PC-A                          PC-B

TCP connection
──────────────────────────────────►

Source Port = 50000
Destination Port = 443
```

The router forwards the IP packet containing the TCP segment.

Normally:

```
TCP source port
TCP destination port
TCP sequence
TCP ACK
TCP payload
```

are not changed merely because a normal router forwards the packet.

However, features such as NAT, firewalls, proxies, or load balancers can inspect or modify higher-layer information.

---

26. Complete Layer-by-Layer Encapsulation

This is another diagram you should memorize.

Starting with:

```
Application Data
```

Layer 4

```
TCP Header
+
Application Data

       ↓

TCP Segment
```

Layer 3

```
IP Header
+
TCP Segment

       ↓

IP Packet
```

Layer 2

```
Ethernet Header
+
IP Packet
+
FCS

       ↓

Ethernet Frame
```

Layer 1

```
Ethernet Frame

       ↓
```

So:

```
APPLICATION DATA
       ↓
TCP SEGMENT
       ↓
IP PACKET
       ↓
ETHERNET FRAME
       ↓
BITS ON WIRE
```

---

27. What Happens at PC-B?

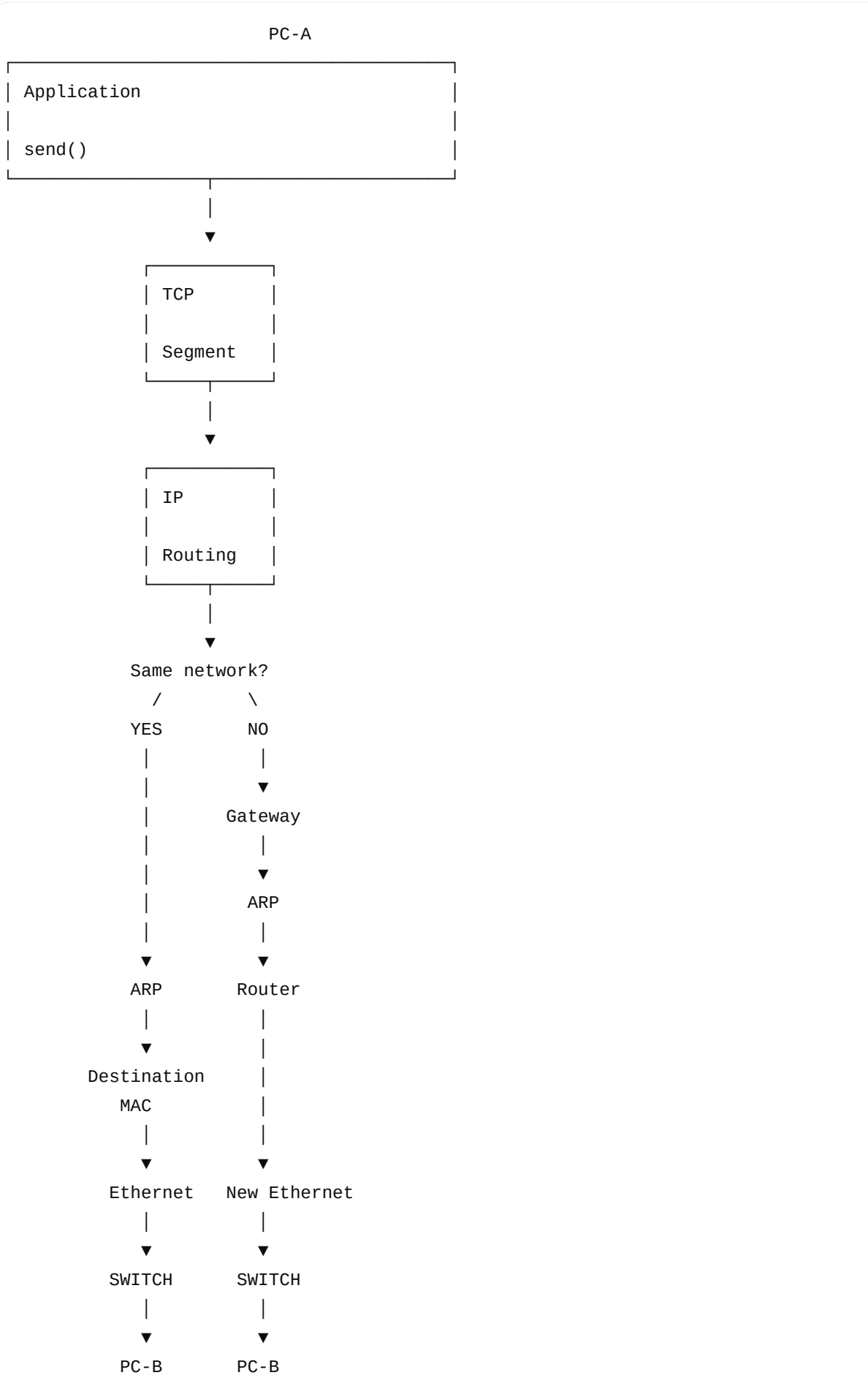
The reverse happens.

```
Bits
 |
 ▼
NIC
 |
 ▼
Ethernet Frame
 |
 | Remove Layer 2 header
 ▼
IP Packet
 |
 | Check destination IP
 ▼
TCP Segment
 |
 | Check destination port
 ▼
Socket
 |
 ▼
Application
```

This is called **decapsulation**.

---

28. Complete PC-A → PC-B


PC-A
Application
send()
TCP
Segment
IP
Routing
Same network?
YES
NO
Gateway
ARP
ARP
Router
Destination
MAC
Ethernet
New Ethernet
SWITCH
SWITCH
PC-B
PC-B

29. Compare Same LAN and Different Network

Concept	Same LAN	Different Network
Destination IP	PC-B	PC-B
Routing lookup	Local route	Remote route/default route
Router	× Not required	✅ Required
ARP target	PC-B	Next-hop router
First destination MAC	PC-B	Router
Switch	✅	✅
MAC changes	No router involved	At every router hop
IP destination	PC-B	PC-B
TCP connection	End-to-end	End-to-end in normal forwarding
Layer 2	Local delivery	New frame per link
Layer 3	Determines local delivery	Determines next hop

---

30. What Each Device Actually Does

This is excellent for interview preparation.

Application

```
Creates/uses application data
Calls socket APIs
```

TCP

```
Ports
Segmentation
Sequence numbers
ACK
Retransmission
Flow/congestion control
```

IP

```
Source/Destination IP
Routing
TTL
Packet forwarding
```

ARP

```
IPv4 address → MAC address
```

More precisely, ARP is a Layer-2/Layer-3 boundary protocol used to resolve IPv4 next-hop addresses to Ethernet MAC addresses on a local network.

Switch

```
MAC address → Port
```

Router

```
Destination IP
      ↓
Routing table
      ↓
Next hop / outgoing interface

AND

rebuilds Layer-2 framing for the outgoing link
```

NIC

```
Kernel packet buffers
       ↓
Driver
       ↓
NIC hardware
       ↓
Physical transmission
```

---

31. Very Important Interview Correction

Don't say:

> × "The router works only on Layer 2."

Say:

> ✅ **"A router primarily operates at Layer 3. It examines the destination IP and routing table to select the next hop/outgoing interface. It removes the incoming Layer-2 frame and creates a new Layer-2 frame for the outgoing network."**

And don't say:

> × "IP is changed at every router."

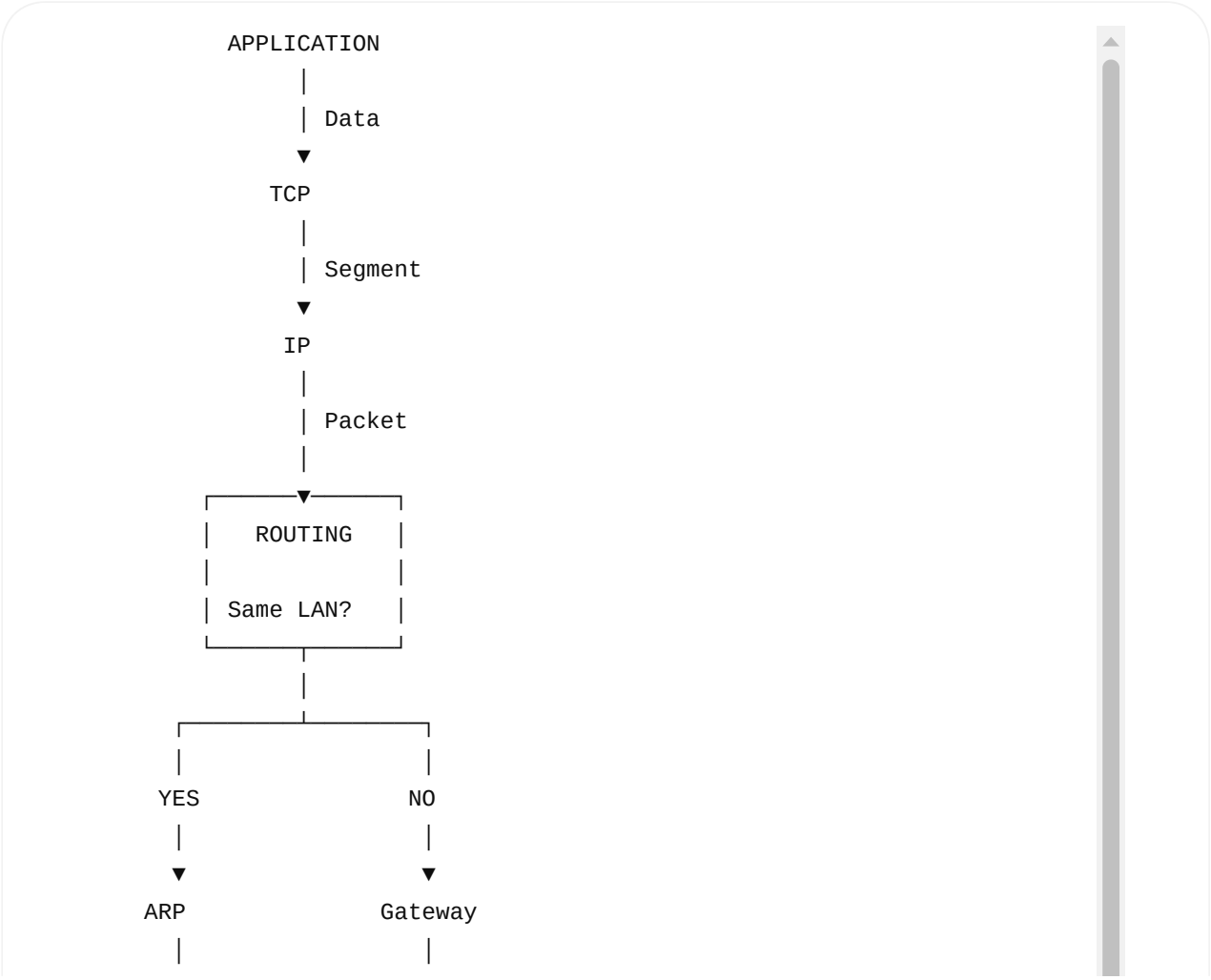
Say:

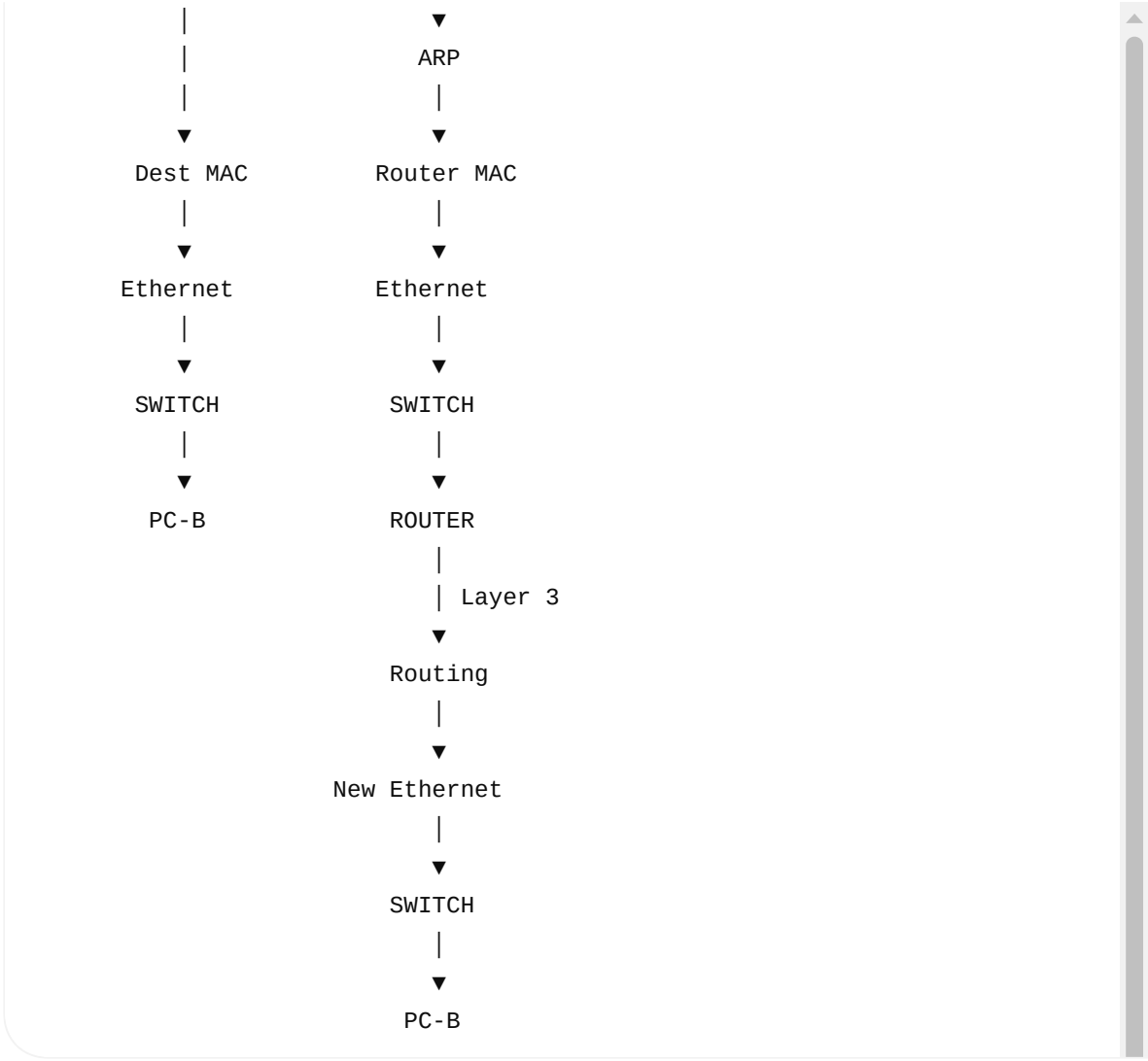
> ✅ **"The Layer-2 MAC addresses change at every routed hop, while the IP addresses normally remain end-to-end, except when functions such as NAT modify them."**

---

32. One Final Mental Model

If you remember nothing else, remember this:





The golden rule 🔑

```
TCP  → "Which application?"
IP   → "Where is the destination?"
ARP  → "What MAC is my next hop?"
MAC  → "Which local device?"
Switch → "Which port?"
Router → "Which next network?"
```

And the most important relationship:

```
          IP = END-TO-END
               │
               ▼
PC-A ──────────────── PC-B


        MAC = HOP-BY-HOP
               │
               ▼
PC-A → Switch → Router → Switch → PC-B
```

Once this model is clear, **Layer 1, Layer 2, Layer 3, Layer 4, switches, routers, ARP, TCP, and Linux socket programming all start fitting together instead of feeling like separate topics.**